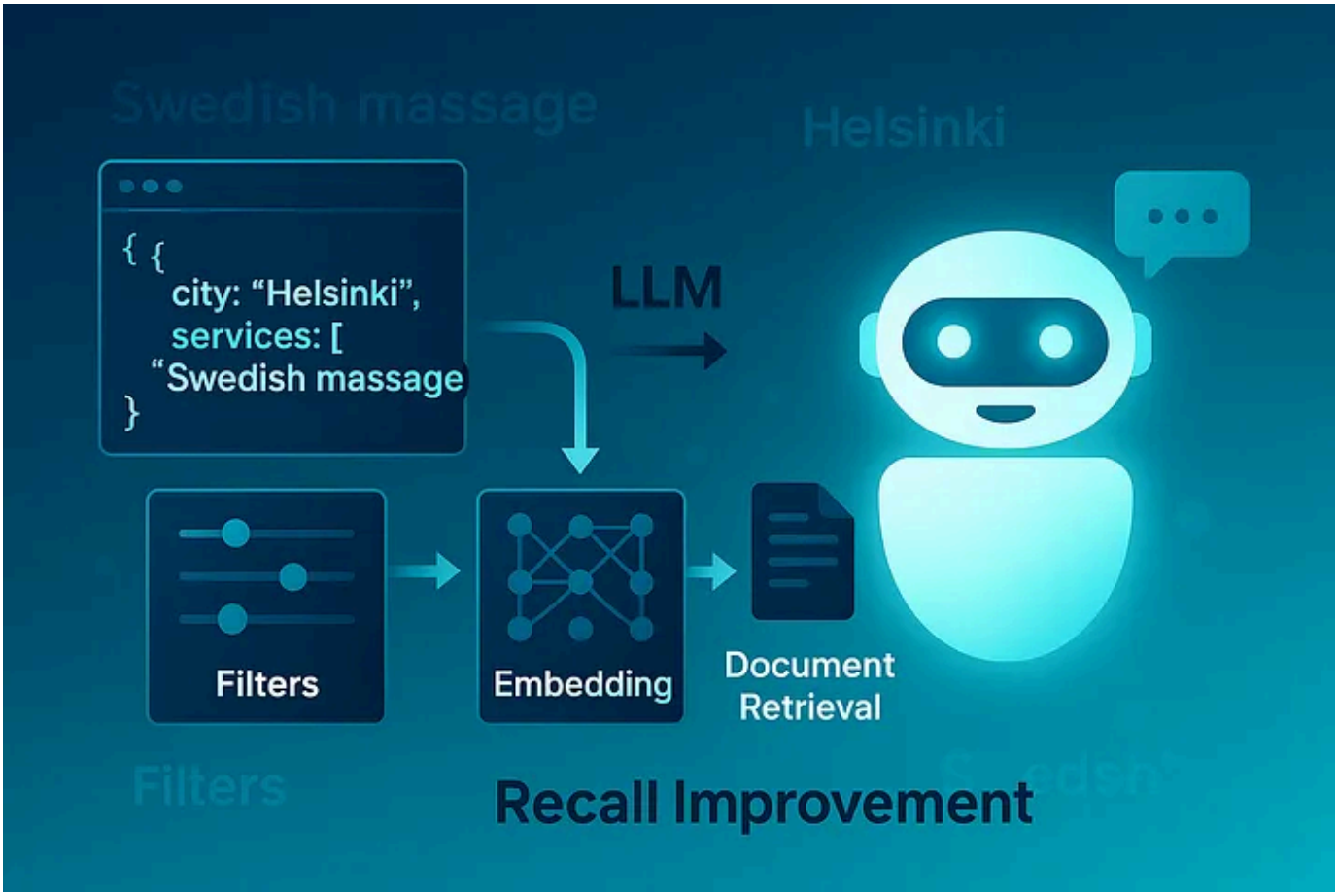
How I Boosted RAG Retrieval Recall from 50% to Over 95%

A simple yet powerful strategy to fix poor document retrieval in RAG pipelines using structured filtering and LLMs

5 min read · 4 days ago

Muhammad Haider Tallal Follow



Made with Ai

For Non Members

In this article, I'm going to teach you a few great techniques for improving retrieval in RAG (Retrieval-Augmented Generation) applications. I used these recently in a client project to increase our system's recall from around 50–60% all the way to 95% and above.

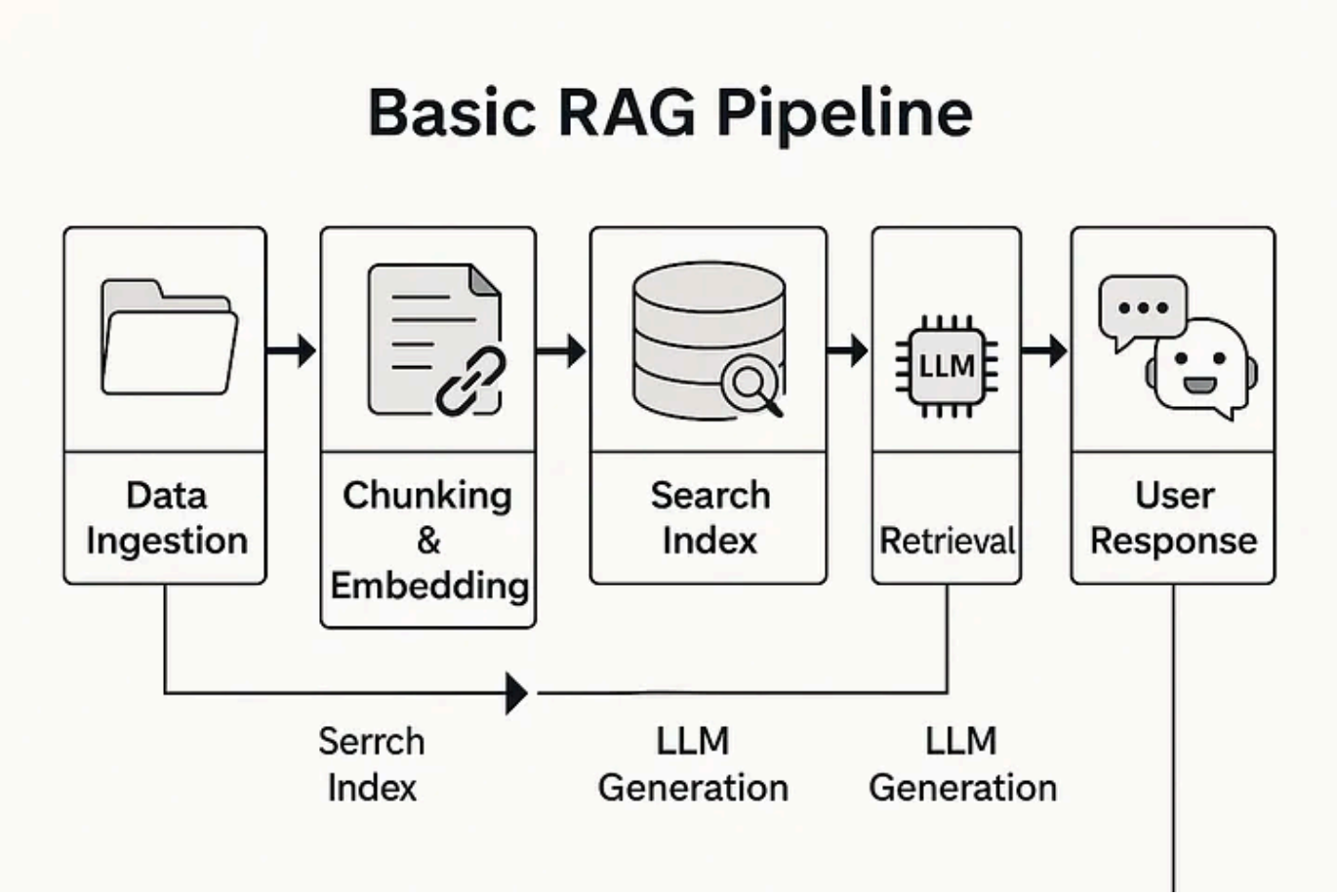
Recall is a very important metric for any RAG application. It simply measures how well a retrieval system finds the correct documents for a given user search or question. That, of course, strongly impacts how good our generated results are — whether we're building a chatbot or something similar.

Most people know this by now: if you don't give the necessary context to the LLM, it can't generate good results, no matter how much prompt engineering you do at the end of the chain. So in RAG, the biggest factor — often by far — in getting good results is **good retrieval**. And one way we measure that is **recall**.

Now, I'll walk you through this client project in a bit of detail so you can see exactly what we did and how we achieved over 95% recall — and therefore a very reliable system.

• • •

The Setup: A Classic RAG Pipeline



The Setup: A Classic RAG Pipeline | Made With AI

At a high level, this was a pretty classic RAG project. We built a chatbot for internal use — specifically for customer service staff to find information faster.

The basic flow:

- Fetch data from different customer systems and databases.
- Preprocess the data: chunking, embedding, and all the usual RAG stuff.
- Build a search index from those document chunks.
- Connect the chatbot to that search index so users can ask questions in natural language.

The bot retrieves relevant documents from the index, then uses that to generate responses. Simple, standard RAG.

. . .

Initial Version: The Naive Approach

Our first naive version looked something like this:

On the indexing side:

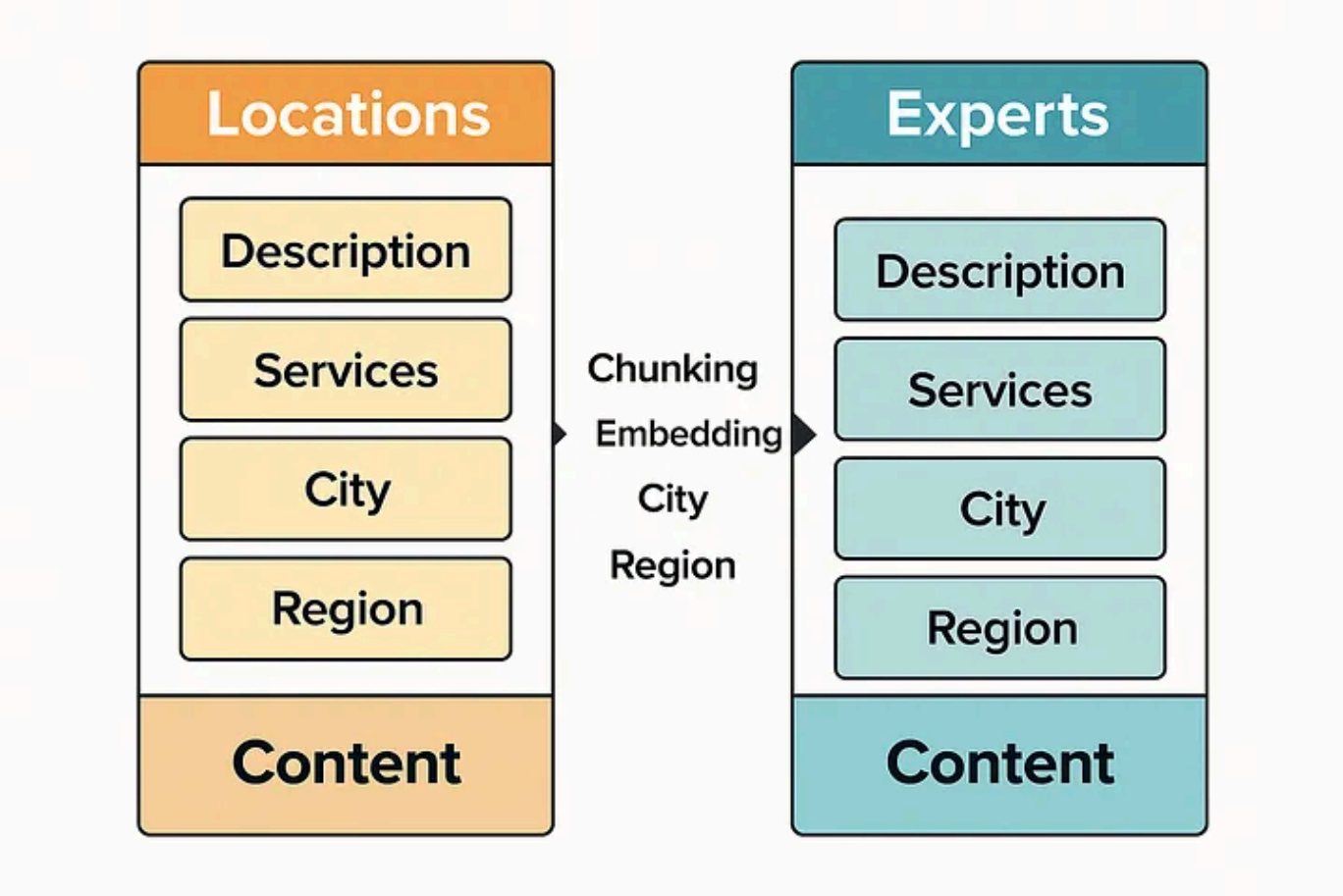
- We cleaned up the data from different systems.
- Chunked it into smaller pieces.
- Created embeddings for vector search.
- Loaded those chunks and embeddings into a vector database (Azure AI Search in this case).

We had all kinds of data, but for this example, we'll focus on two document types: **locations** and **experts**.

This client was in the spa and wellness space. They had:

- **Locations** like spas and gyms, each with descriptions including services (e.g. treatments, massages), machines, city, and region.
- **Experts** like massage therapists and personal trainers, with similar service descriptions.

We joined all the relevant fields (description, city, region) into one **content** field for text search. We also created embeddings of that field for vector search.



visual comparing 'Locations' and 'Experts' as unified content documents — designed for clarity and clean indexing | made with ai

On the frontend:
Users would type something like:

> "Swedish massage in Helsinki"

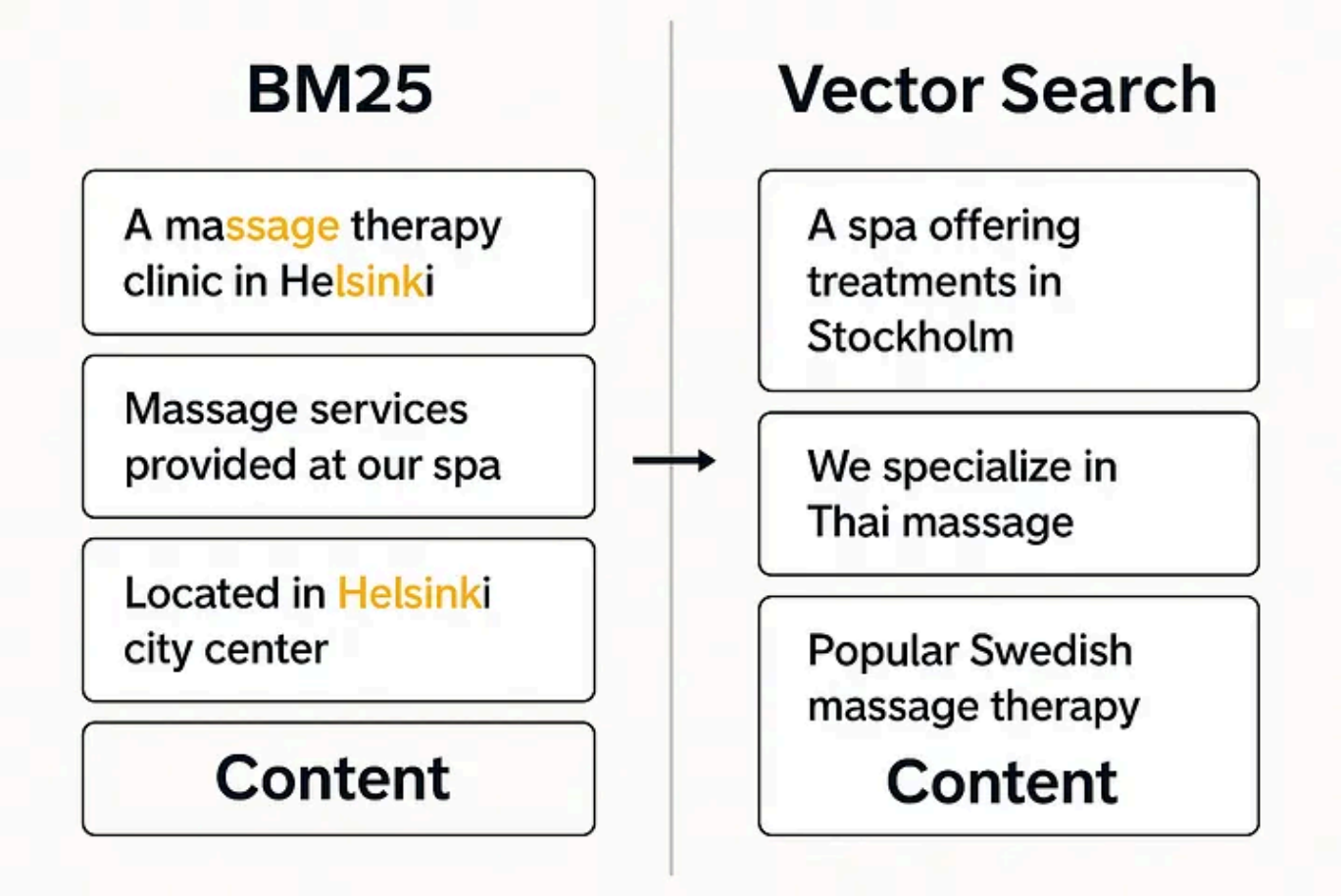
Then we'd run that query as either:

- A **vector search** against the content vector field, or
- A **full-text BM25 search** against the content field.

We tried both — but both had problems.

• • •

Why It Didn't Work



Compare BM25 vs vector search results made with ai

Vector Search

This was a total no-go.

While vector search is great for fuzzy matching and semantic similarity, in our case we needed **exact matches** — for both **services** and **locations**.

Instead, vector search would return similar services or cities (like other massages or other capitals in Finland) but not exactly what the user asked for. Not helpful.

BM25

Slightly better, but still not good.

BM25 ranks documents based on the frequency of search terms. That sounds okay until you realize that:

- A document with many mentions of "massage" and a random "Swedish meatballs" mention might rank **higher** than an actual Helsinki spa that does **Swedish massage** — just because of term frequency.

It doesn't prioritize **exact matches**, which was our main need.

Other Issues

We also ran into:

- **Conjugation issues** — especially since the project was in Finnish. For example, "in Helsinki" is conjugated in different ways, and if the conjugation didn't match the user query exactly, BM25 wouldn't find it

. . .

The Fix: Structured Search with LLM Assistance

Here's how we fixed it and boosted recall to over 95%.

Step 1: Modify the Index

We added a new field to the search index: **services**, as a structured **list** rather than embedding them in the freeform description.

But this data wasn't directly available — so we used an LLM to extract services during indexing.

For example, from a location or expert description, we'd prompt the LLM to generate:

```
services: ["Swedish massage", "facial", "deep tissue massage"]
```

We then removed the vector embeddings altogether — they weren't useful for our needs.

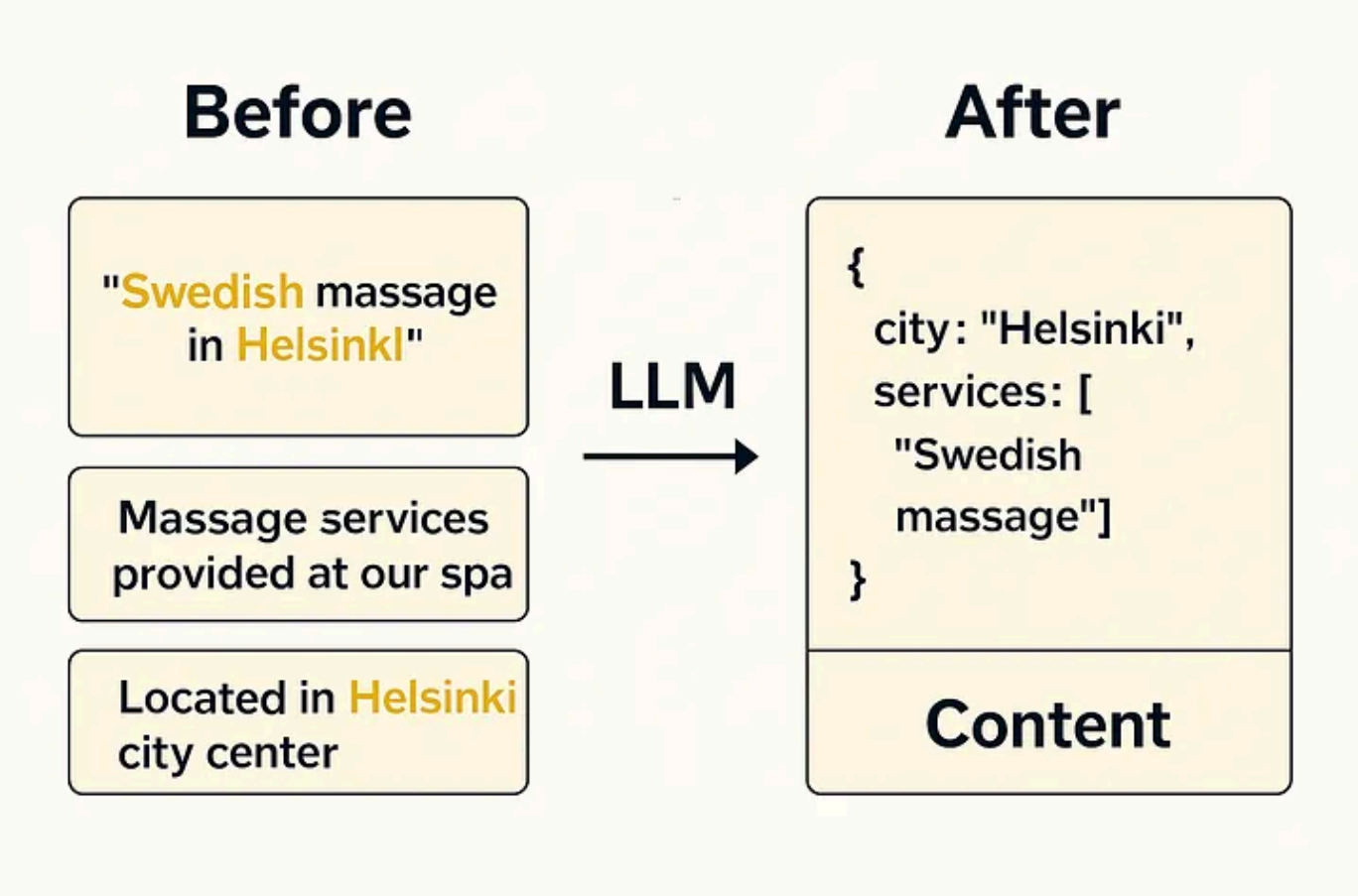
Step 2: Transform the Query

This was the real game-changer.

Instead of passing the user's raw query directly into search, we now used an LLM to **structure the query** into a format like this:

```
{
  "city.fi": "Helsinki",
  "services": ["Swedish massage"]
}
```

That way, we could run a **precise filter** query on city and services fields — getting only the documents that matched exactly.



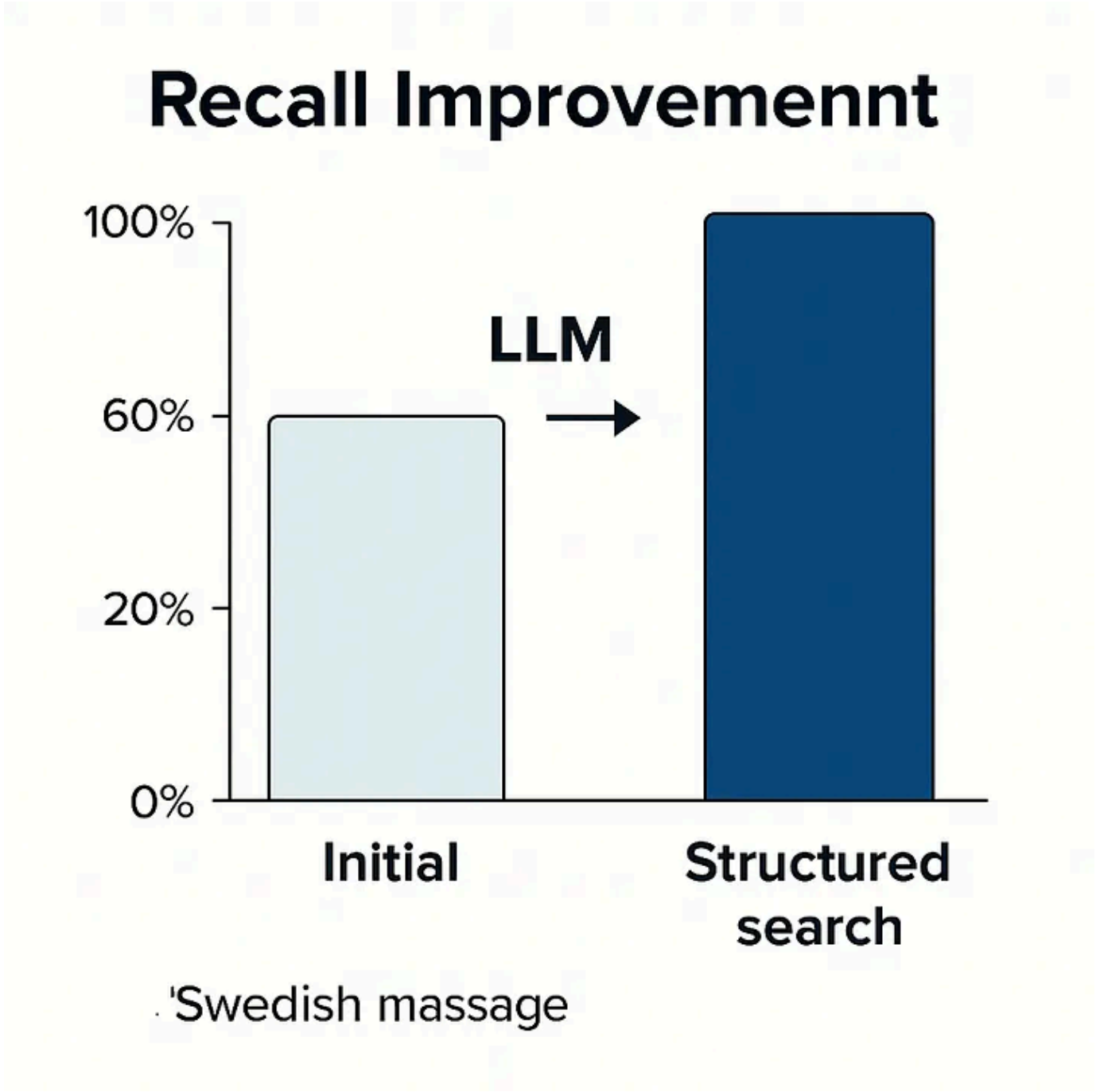
Show how query is transformed and indexed with structure made with ai

. . .

The Results

We ran another round of user testing after implementing these changes, and the results were clear:

- **Recall jumped from 50–60% to nearly 100%.**
- Most previous issues were resolved.
- Only a few edge cases remained, mostly due to poor data quality.



Display the recall improvement visually made with ai

. . .

Trade-Offs

Of course, there were a few trade-offs:

1. **Indexing is now more expensive**, since we use an LLM to extract services. But this tool was built for hundreds of internal users — saving them thousands of hours — so it was well worth the cost.
2. **Slight latency** on the frontend. We added one extra LLM interaction to structure the query before retrieval. But it's fast: short inputs and outputs, and we used a small model here.

. . .

Final Thoughts

This was a big win, achieved with relatively simple and intuitive changes.

Sometimes you don't need agentic RAG or other trendy techniques from undergrad research papers. You just need to clearly understand your actual issues.

In our case, users needed **exact** matches for specific service-location queries. That pointed us toward **structured filtering** as the solution.

And while RAG usually means retrieval **augments** generation, it also works the other way around. There are all kinds of clever ways to use LLMs to build **better retrieval**.

Hope that was helpful — and I'll see you in the next one.

. . .

> *By Muhammad Haider Tallal*